NLV / CX3 HPC Autonomy Cheatsheet

Imperial CX3 + PBSPro + the current **nlv-hpc** package. Designed as an end-to-end operating manual: local VSCode/PowerShell, SSH, code sync, pre-flight checks, submission, monitoring, post-run diagnosis, timing, collection, and export.

Project assumptions. HPC username: **vl1324**. Remote project: **~/nlv-hpc**. Local project: **C:\Users\pengl\Research\Exposition-to-NDEs\Codes\nlv-hpc**. Scheduler: PBSPro on CX3. Conda environment: **nlv-hpc**.

Most important mental model: your Windows folder and **~/nlv-hpc** are separate copies. Editing a local file does not alter the HPC copy until you upload it.

For your completed runs right now: skip directly to Section 7. qstat is expected to show nothing once jobs have finished.

1. Know which terminal you are in

Use two distinct terminal roles. Do not mix them.

Terminal	Typical prompt	Use it for
LOCAL VSCode PowerShell	PS C:\Users\pengl\...\nlv-hpc>	SSH, SCP uploads/downloads, local hashes, opening downloaded results.
HPC login shell	(nlv-hpc) [vl1324@login-bi nlv-hpc]$	Conda, checks, PBS submission, qstat, logs, collectors, result inspection.

Local PowerShell: define shortcuts once per session

```
$HPC   = "vl1324@login.cx3.hpc.imperial.ac.uk"
$LOCAL = "C:\Users\pengl\Research\Exposition-to-NDEs\Codes\nlv-hpc"
cd $LOCAL
```

Log in to CX3

```
ssh $HPC
```

If SSH cannot connect, first check that you are on the Imperial network or the approved remote-access path. Once connected, hostname will normally show a CX3 login node such as login-bi.

2. Start every HPC session cleanly

```
cd ~/nlv-hpc

set +u
eval "$(~/miniforge3/bin/conda shell.bash hook)"
conda activate nlv-hpc

pwd
hostname
echo "$CONDA_DEFAULT_ENV"
which python
python --version
```

Expected: project directory is .../nlv-hpc, environment is nlv-hpc, and Python points inside Miniforge.

Do not rerun environment setup every session. bash -l setup_environment.sh is a first-time/rebuild operation, not a normal login command.

First-time environment setup only

```
cd ~/nlv-hpc
bash -l setup_environment.sh
```

The current package creates Python 3.11 with pinned numerical packages and PyTorch/CUDA dependencies.

3. Sync code safely between VSCode and HPC

Upload one edited source file from LOCAL PowerShell

```
scp "$LOCAL\nlv_core.py" "${HPC}:~/nlv-hpc/nlv_core.py"
```

Upload several edited source files

```
scp "$LOCAL\nlv_core.py" "$LOCAL\run_nlv.py" "$LOCAL\submit_jobs.py" `
    "${HPC}:~/nlv-hpc/"
```

Verify local and remote files are identical

LOCAL PowerShell:

```
Get-FileHash "$LOCAL\nlv_core.py" -Algorithm SHA256
```

HPC terminal:

```
cd ~/nlv-hpc
sha256sum nlv_core.py
```

The SHA256 values must match. This is the strongest simple check that you are running the file you actually edited.

Freeze production source after submission. The dispatcher records hashes of the model/job source at submission. Changing those files while jobs are queued causes those queued jobs to stop with a source-change error.

Current run_identity patch sanity check

```
grep -n "def run_identity" -A55 nlv_core.py
grep -n "payload = json.loads(json.dumps(payload, sort_keys=True))" nlv_core.py
```

For the current fix, the second command should find the JSON canonicalisation line. The run-id hash should still use ordinary json.dumps(payload, sort_keys=True) formatting.

4. Five-minute pre-flight before spending GPU time

Run these on the HPC login node. They do not perform the expensive calibration.

```
cd ~/nlv-hpc

# Python syntax.
python -m py_compile \
    nlv_core.py run_nlv.py submit_jobs.py collect_results.py jobs/dispatch_job.py

# Shell/PBS syntax.
bash -n jobs/job_common.sh setup_environment.sh jobs/*.pbs

# See all named model configurations.
python run_nlv.py --list-configs

# Check required data files, CSV headers, and CPU anchor presence.
python run_nlv.py --mode development --check-inputs

# Confirm the exact first-wave list.
cat configs/ablations.txt

# Validate and display the array qsub command without submitting.
python submit_jobs.py --kind array --concurrency 4 --skip-reports --dry-run
```

Interpretation: no output from py_compile or bash -n means success. The input check should explicitly say it passed. The dry run should list the intended configurations and a PBS command.

Optional storage check before a large wave

```
du -sh data runs logs 2>/dev/null
du -sh runs/development/* 2>/dev/null | sort -h
```

5. Submit jobs

Smoke

```
python submit_jobs.py --kind smoke
```

Reference

```
python submit_jobs.py --kind reference --skip-reports
```

First-wave ablation array

```
python submit_jobs.py --kind array --concurrency 4 --skip-reports
```

Use a longer walltime

```
python submit_jobs.py \
    --kind array \
    --concurrency 4 \
    --walltime 24:00:00 \
    --skip-reports
```

Submit another configuration file

```
python submit_jobs.py \
    --kind array \
    --config-file configs/ablations_all.txt \
    --concurrency 4 \
    --skip-reports
```

Finalist confirmation on new seeds

```
python submit_jobs.py \
    --kind array \
    --config-file configs/finalists_example.txt \
    --seed 22345 \
    --concurrency 4 \
    --skip-reports

python submit_jobs.py \
    --kind array \
    --config-file configs/finalists_example.txt \
    --seed 32345 \
    --concurrency 4 \
    --skip-reports
```

Optional: make a later job wait for another PBS job

```
python submit_jobs.py \
    --kind reference \
    --afterok JOB_ID \
    --skip-reports
```

The submitter creates an immutable snapshot under jobs/submissions/. Always submit through submit_jobs.py; do not call the PBS templates directly.

Find the latest submission snapshot and job ID

```
latest=$(ls -dt jobs/submissions/* | head -1)
echo "$latest"
cat "$latest/job_id.txt"
cat "$latest/submission.json"
```

6. Monitor queued and running jobs

Normal queue view

```
qstat -u "$USER" -t
```

Common states: **Q** queued, **R** running, **B** array with active subjobs, **X** completed/deleted array subjob. Completed jobs eventually disappear from the live queue.

Refresh the queue every 30 seconds

```
watch -n 30 'qstat -u "$USER" -t'
```

Press Ctrl+C to stop watch. It does not cancel any jobs.

Detailed scheduler information for one queued/running job

```
qstat -f JOB_ID | less
```

Show assigned compute node while a job is running

```
qstat -nu "$USER" -1
```

The last column contains the compute node. This is useful for GPU debugging.

Optional: SSH to the assigned compute node while your job is running

```
export PBS_JOBID=JOB_ID
ssh COMPUTE_NODE
```

Use this only for diagnosis. Commands such as nvidia-smi, top, or htop can confirm activity. Do not start a second calibration manually on the compute node.

Cancel a queued/running job

```
qdel JOB_ID
```

7. Jobs finished and qstat is empty: inspect results

This is your current situation. An empty qstat -u "$USER" after the jobs have run is normal. Use the project logs and result metadata, not qstat, for completed jobs.

7.1 Find the newest logs

```
cd ~/nlv-hpc
ls -lt logs/*.log | head -20
```

7.2 Read the end of every log from one PBS array

```
tail -n 100 logs/*ARRAY_JOB_NUMBER*
```

Example: for PBS array 4269891, use tail -n 100 logs/*4269891*.

7.3 Search all logs for success/failure signatures

```
grep -HnE \
'Completed\. Results:|Traceback|RuntimeError|failed|interrupted|source-change|changed after submission' \
logs/*.log | tail -200
```

7.4 Print every run status in one compact table

```
python - <<'PY'
from pathlib import Path
import json

for path in sorted(Path("runs").rglob("hpc_status.json")):
    d = json.loads(path.read_text())
    err = d.get("error", "")
    print(
        f"{d.get('mode','?'):11} "
        f"{d.get('config','?'):26} "
        f"seed={d.get('seed','?'):>10} "
        f"status={d.get('status','?'):12} "
        f"job={d.get('pbs_job_id','-')}"
    )
    if err:
        print("  error:", err)
PY
```

A successful research run should have all three: outer hpc_status.json with status completed; a hashed run folder containing completed.json; and daily_summary.csv.

7.5 List completed result files

```
find runs/development -name completed.json -print
find runs/development -name daily_summary.csv -print
```

7.6 Inspect one completed.json cleanly

```
python -m json.tool PATH/TO/completed.json | less
```

7.7 Inspect recent training-history files

```
find runs/development -name '*_history.csv' -printf '%T@ %p\n' \
  | sort -nr | head -20
```

Then inspect one:

```
tail -n 10 PATH/TO/rolling_YYYYMMDD_history.csv
```

8. How long did every job take?

There are two useful timings. **Total runner time** comes from hpc_status.json (start to finish, including audits and reporting if enabled). **Core experiment time** comes from completed.json: wall_time_seconds and covers the numerical experiment itself.

One command: status, PBS ID, end-to-end time, and core time

```
python - <<'PY'
from pathlib import Path
from datetime import datetime
import json

def dt(s):
    return datetime.fromisoformat(s.replace("Z", "+00:00"))

rows = []
for status_path in Path("runs").rglob("hpc_status.json"):
    s = json.loads(status_path.read_text())
    start = s.get("started_utc")
    finish = s.get("finished_utc")
    total = (dt(finish) - dt(start)).total_seconds() if start and finish else None

    core = None
    folder = s.get("folder")
    if folder:
        completed = Path(folder) / "completed.json"
        if completed.is_file():
            core = json.loads(completed.read_text()).get("wall_time_seconds")

    rows.append((
        s.get("mode",""),
        s.get("config",""),
        s.get("seed",""),
        s.get("status",""),
        s.get("pbs_job_id",""),
        total,
        core,
    ))

print(f"{'MODE':11} {'CONFIG':26} {'SEED':10} {'STATUS':12} {'TOTAL(h)':>10} {'CORE(h)':>10}  JOB")
for mode, cfg, seed, status, job, total, core in sorted(rows):
    total_h = "-" if total is None else f"{total/3600:.3f}"
    core_h  = "-" if core  is None else f"{core/3600:.3f}"
    print(f"{mode:11} {cfg:26} {str(seed):10} {status:12} {total_h:>10} {core_h:>10}  {job}")
PY
```

Include approximate queue waiting time using submission snapshots

```
python - <<'PY'
from pathlib import Path
from datetime import datetime
import json

def dt(s):
    return datetime.fromisoformat(s.replace("Z", "+00:00"))

def base(job):
    if not job:
        return None
    return job.split(".", 1)[0].split("[", 1)[0]

submitted = {}
for meta in Path("jobs/submissions").glob("*/submission.json"):
    jid = meta.parent / "job_id.txt"
    if not jid.is_file():
        continue
    m = json.loads(meta.read_text())
    submitted[base(jid.read_text().strip())] = dt(m["created_utc"])

print(f"{'CONFIG':26} {'QUEUE(h)':>10} {'RUNNER(h)':>10}  JOB")
for path in sorted(Path("runs").rglob("hpc_status.json")):
    s = json.loads(path.read_text())
    job = s.get("pbs_job_id")
    start = s.get("started_utc")
    finish = s.get("finished_utc")
    if not start:
        continue
    st = dt(start)
    sub = submitted.get(base(job))
    queue_h = (st - sub).total_seconds()/3600 if sub else None
    run_h = (dt(finish)-st).total_seconds()/3600 if finish else None
    q = "-" if queue_h is None else f"{queue_h:.3f}"
    r = "-" if run_h is None else f"{run_h:.3f}"
    print(f"{s.get('config','?'):26} {q:>10} {r:>10}  {job}")
PY
```

Queue time here is measured from the package's saved submission timestamp to the point where run_nlv.py starts on the allocated node. It is an operational estimate and is usually the useful number for this workflow.

9. Collect and compare completed research runs

Build comparison outputs

```
cd ~/nlv-hpc
python collect_results.py --runs-root runs
```

The collector performs no calibration. It validates completed runs and writes the comparison tables under runs/comparison/.

List comparison files

```
ls -lh runs/comparison/
```

Key files: completed_runs.csv, summary_by_configuration.csv, daily_comparison.csv, paired_reference_changes.csv, paired_daily_changes.csv, pareto_frontier.csv, excluded_runs.csv, comparison_manifest.json.

Check whether anything was excluded

```
cat runs/comparison/excluded_runs.csv
```

If it contains only the header, nothing was excluded. If rows appear, read the reason column before comparing configurations.

Print the main summary in a readable narrow table

```
python - <<'PY'
import pandas as pd

p = "runs/comparison/summary_by_configuration.csv"
d = pd.read_csv(p)

cols = [
    "name",
    "seed_set",
    "otm_pricing_RMSE_mean",
    "otm_learned_cv_hedge_RMSE_mean",
    "otm_frozen_physical_hedge_RMSE_mean",
    "otm_sticky_normalised_hedge_RMSE_mean",
    "wall_time_seconds_mean",
]
print(
    d[cols]
    .sort_values("otm_pricing_RMSE_mean")
    .to_string(index=False)
)
PY
```

Print changes versus gpu_reference_8k

```
python - <<'PY'
import pandas as pd

p = "runs/comparison/paired_reference_changes.csv"
d = pd.read_csv(p)

cols = [
    "name",
    "seed",
    "otm_pricing_RMSE_change_pct",
    "otm_learned_cv_hedge_RMSE_change_pct",
    "otm_frozen_physical_hedge_RMSE_change_pct",
    "wall_time_seconds_change_pct",
]
print(d[cols].to_string(index=False))
PY
```

For error metrics, negative percentage changes mean improvement relative to the same-seed reference. Compare only within the same collector cohort and seed set.

Inspect the Pareto frontier

```
python - <<'PY'
import pandas as pd
d = pd.read_csv("runs/comparison/pareto_frontier.csv")
print(d[[
    "name",
```

```
    "seed_set",
    "otm_pricing_RMSE_mean",
    "otm_learned_cv_hedge_RMSE_mean",
    "otm_frozen_physical_hedge_RMSE_mean",
]].to_string(index=False))
PY
```

10. Download results into your local VSCode project

Run these from **LOCAL PowerShell**, not from the HPC shell.

Create a local export directory

```
$HPC    = "vl1324@login.cx3.hpc.imperial.ac.uk"
$EXPORT = "C:\Users\pengl\Research\Exposition-to-NDEs\Codes\nlv-hpc-exports"

New-Item -ItemType Directory -Force $EXPORT | Out-Null
```

Download the comparison tables

```
scp -r "${HPC}:~/nlv-hpc/runs/comparison" "$EXPORT\comparison"
```

Download all job logs

```
scp -r "${HPC}:~/nlv-hpc/logs" "$EXPORT\logs"
```

Download submission snapshots/provenance

```
scp -r "${HPC}:~/nlv-hpc/jobs/submissions" "$EXPORT\submissions"
```

Download one full configuration/seed result

```
scp -r `
  "${HPC}:~/nlv-hpc/runs/development/CONFIG_NAME/seed_12345" `
  "$EXPORT\CONFIG_NAME-seed_12345"
```

Recommended: download comparison/, logs/, and submission snapshots after every wave. Download full hashed run folders only for configurations you need to analyse deeply; contract-level CSVs and checkpoints can be much larger.

Open the downloaded export folder in VSCode

```
code $EXPORT
```

If code is not on your PowerShell PATH, open the folder through VSCode normally.

11. Resume, rerun, or extend experiments

Walltime interruption: resume with unchanged scientific identity

The training code checkpoints completed updates. If a job hits walltime, repeat the same submission with the same source, configuration, seed, data, CPU anchor, and environment, optionally requesting a longer walltime.

```
python submit_jobs.py \
    --kind array \
    --config-file configs/ablations.txt \
    --concurrency 4 \
    --walltime 24:00:00 \
    --skip-reports
```

Run one named development configuration

```
python submit_jobs.py \
    --kind reference \
    --config CONFIG_NAME \
    --seed 12345 \
    --skip-reports
```

Before a new wave, edit only the next config list

```
nano configs/ablations.txt
cat configs/ablations.txt
python submit_jobs.py --kind array --dry-run --concurrency 4 --skip-reports
```

After the real submission, freeze the production source until that wave finishes.

12. Fast diagnosis by symptom

Symptom	First command	Meaning / next action
Job stays Q	qstat -f JOB_ID	Usually scheduler/resource wait. Do not assume code failure.
Job disappeared from qstat	ls -lt logs/*.log | head	Normal after finish; inspect logs/status files.
Log has Traceback	tail -n 120 LOG	Read the final exception first; do not resubmit blindly.
status=failed	python status-table command	Read hpc_status.json error plus matching log.
status=completed	find ... completed.json	Verify numerical completion exists, then collect.
Collector excludes run	cat runs/comparison/excluded_runs.csv	Usually identity/cohort/protocol mismatch or duplicate.
Source-change error	cat latest submission.json	A production file changed after submission; submit a fresh snapshot.
Another job writing output	qstat -u "$USER" -t	Duplicate concurrent config/seed/mode. Let one job own the directory.
No progress in log	find ... '*_history.csv'	Progress bars are suppressed by default; history/checkpoints are better evidence.

Search only one specific array's logs for failures

```
grep -HnE 'Traceback|RuntimeError|failed|interrupted|Completed\. Results:' \
    logs/*ARRAY_JOB_NUMBER*
```

13. Rules that prevent wasted nights

- **Never assume the local Windows file is the HPC file.** Upload, then verify SHA256.
- **Never edit production source after submitting a wave.** The dispatcher intentionally rejects changed source.
- **Never submit the PBS template directly.** Use submit_jobs.py so the immutable snapshot exists.
- **Never interpret an empty qstat as missing results.** Finished jobs disappear; inspect logs/status/completed files.
- **Never compare runs that the collector places in different cohorts.** Runtime/source/protocol differences are deliberately separated.
- **Never delete old run directories before collection and export.** They contain checkpoints, contract-level outputs, and provenance.
- **Never launch heavy calibration on the login node.** Login-node commands should be checks, submission, collection, and light file inspection.
- **Before an overnight wave, run syntax checks, input checks, and a dry run.** These catch cheap errors before queue time.

14. One-screen standard workflow

```
# ========== LOCAL POWERSHELL ==========
$HPC   = "vl1324@login.cx3.hpc.imperial.ac.uk"
$LOCAL = "C:\Users\pengl\Research\Exposition-to-NDEs\Codes\nlv-hpc"
cd $LOCAL

# Upload edited source if needed.
scp "$LOCAL\nlv_core.py" "${HPC}:~/nlv-hpc/nlv_core.py"

# Log in.
ssh $HPC


# ========== HPC LOGIN NODE ==========
cd ~/nlv-hpc
set +u
eval "$(~/miniforge3/bin/conda shell.bash hook)"
conda activate nlv-hpc
```

```
# Cheap pre-flight.
python -m py_compile nlv_core.py run_nlv.py submit_jobs.py collect_results.py jobs/dispatch_job.py
bash -n jobs/job_common.sh setup_environment.sh jobs/*.pbs
python run_nlv.py --mode development --check-inputs
cat configs/ablations.txt
python submit_jobs.py --kind array --concurrency 4 --skip-reports --dry-run

# Submit.
python submit_jobs.py --kind array --concurrency 4 --skip-reports

# Monitor.
qstat -u "$USER" -t
ls -lt logs/*.log | head
tail -f "$(ls -t logs/*.log | head -1)"

# After completion.
grep -HnE 'Completed\. Results:|Traceback|RuntimeError|failed|interrupted' logs/*.log | tail -200
python collect_results.py --runs-root runs
ls -lh runs/comparison
cat runs/comparison/excluded_runs.csv


# ========== LOCAL POWERSHELL AGAIN ==========
$EXPORT = "C:\Users\pengl\Research\Exposition-to-NDEs\Codes\nlv-hpc-exports"
New-Item -ItemType Directory -Force $EXPORT | Out-Null
scp -r "${HPC}:~/nlv-hpc/runs/comparison" "$EXPORT\comparison"
scp -r "${HPC}:~/nlv-hpc/logs" "$EXPORT\logs"
scp -r "${HPC}:~/nlv-hpc/jobs/submissions" "$EXPORT\submissions"
code $EXPORT
```

15. Your current completed wave: commands to run now

Because the jobs are finished and qstat is empty, run this sequence on the HPC terminal now:

```
cd ~/nlv-hpc
set +u
eval "$(~/miniforge3/bin/conda shell.bash hook)"
conda activate nlv-hpc

# 1. Find logs.
ls -lt logs/*.log | head -20

# 2. Scan all logs for final outcomes.
grep -HnE \
'Completed\. Results:|Traceback|RuntimeError|failed|interrupted' \
logs/*.log | tail -200

# 3. See completed/failed status by configuration.
python - <<'PY'
from pathlib import Path
import json
for p in sorted(Path("runs").rglob("hpc_status.json")):
    d=json.loads(p.read_text())
    print(d.get("config"), d.get("seed"), d.get("status"),
          d.get("pbs_job_id"), d.get("error",""))
PY

# 4. Collect numerical comparisons.
python collect_results.py --runs-root runs

# 5. Inspect collector outputs.
ls -lh runs/comparison
cat runs/comparison/excluded_runs.csv

# 6. Print the main comparison.
python - <<'PY'
import pandas as pd
d=pd.read_csv("runs/comparison/summary_by_configuration.csv")
cols=[
    "name","seed_set",
    "otm_pricing_RMSE_mean",
    "otm_learned_cv_hedge_RMSE_mean",
    "otm_frozen_physical_hedge_RMSE_mean",
    "wall_time_seconds_mean",
]
print(d[cols].sort_values("otm_pricing_RMSE_mean").to_string(index=False))
PY
```

Then use Section 10 from LOCAL PowerShell to download the comparison directory, logs, and submission snapshots.

16. Current package CLI reference

Command/flag	Purpose
--kind smoke	Tiny engineering/numerical test; separate smoke results.
--kind reference	One development configuration; default gpu_reference_8k.
--kind array	PBS array from a configuration text file.
--config NAME	Single configuration name for smoke/reference.
--config-file PATH	Configuration list for array jobs.
--seed N	Random seed; default 12345.
--concurrency N	Maximum simultaneous array subjobs; default 4.
--gpu-type L40S|RTX6000|A100	Requested GPU type; default L40S.
--walltime HH:MM:SS	Default 01:00:00 smoke, 12:00:00 research.
--afterok JOB_ID	PBS success dependency.
--skip-reports	Keep raw numerical exports; skip extra IV reports/plots.
--dry-run	Validate and print qsub command; submit nothing.

17. Basis and reference

Package basis: the supplied **nlv-hpc(1).zip**, including README.md, submit_jobs.py, collect_results.py, run_nlv.py, jobs/job_common.sh, jobs/dispatch_job.py, and PBS templates.

Imperial RCS documentation checked for current CX3 behaviour:

- SSH/login host: https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/getting-started/using-ssh/
- PBS submission/monitoring: https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/getting-started/running-your-first-job/
- PBS arrays: https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/queues/array-jobs/
- Compute-node SSH: https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/faq/ssh-compute-node/
- Job logs: https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/faq/controlling-job-log-files/

Cheatsheet generated 2026-10-05. If Imperial changes scheduler policy or hostnames, recheck the RCS guide.